On-Policy Alignment and Iterative Policy Improvement

pig7selene

September 5, 2026

Abstract

On-policy alignment improves a language-model policy using responses generated by a recent version of that same policy. The apparent simplicity of this idea hides an important systems and statistical contract: every selected response must be traceable to the behavior policy, generation configuration, reward or verifier version, and filtering decision that produced it. This chapter develops the rollout lifecycle, policy-version semantics, exploration and selection, rejection-style policy filtering, and Expert Iteration as an iterative improvement pattern. It then treats DAPO as a concrete modern design, explaining its dynamic sampling, asymmetric clipping, token-level objective, and overlong-response shaping. The emphasis is on how online data, relative rewards, and policy updates form a moving training distribution rather than a static preference dataset.

1 Introduction

Chapters 13–16 introduced a progression from preference data to Reward Models, PPO, DPO, and GRPO. The distinction between offline preference learning and online policy optimization is now central. An offline method can train on a fixed collection of comparisons. An on-policy method instead asks a recent language-model policy to generate candidate responses, scores those responses, and uses them to update the policy that will generate the next collection. The data distribution therefore changes as training proceeds.

This feedback loop is especially useful when a verifier can score newly sampled solutions, as in the reasoning settings of Chapter 17. It is also delicate. A response may be excellent under one policy version but be treated incorrectly after the learner has moved far away from that behavior policy. A filter that improves average reward may remove the very difficult cases required for robust generalization. A rollout worker can produce many candidates, but no amount of throughput repairs an ambiguous reward contract. This chapter focuses on these algorithmic interfaces. The distributed worker topology used to execute them is deferred to Chapter 39.

2 On-Policy Data and Policy Versions

Let π_{θ_v} denote the behavior policy at version v, and let π_θ denote the policy currently being optimized. For a prompt $x \sim \mathcal{D}$, a rollout $y = (y_1, ..., y_T)$ is sampled autoregressively from the behavior policy,

$$y \sim \pi_{\theta_v}(. \mid x) = \prod_{t=1}^{T} \pi_{\theta_v}(y_t \mid x, y_< t). \tag{1}$$

The word **on-policy** means that optimization uses data from the policy being improved, or from a deliberately bounded recent behavior version. By contrast, **off-policy** data comes from a behavior distribution that is not guaranteed to be recent for the update; a fixed preference dataset is a particularly important offline source. A stale rollout is therefore not a separate objective: it is a behavior-policy sample whose version is becoming too distant for the declared on-policy approximation. Practical systems may tolerate limited lag, but they must not silently relabel arbitrary historical data as fresh Rollouts.

This does not mean that a production learner and generator must share one instantaneous set of parameters. In practice, an update may reuse one rollout batch for several optimization epochs, or a generator may finish requests while a learner begins a later update. The essential requirement is that this lag is recorded and controlled.

A useful rollout record contains more than text:

$$r = (x, y, v, \ell_{\text{old}}, m, R, h), \tag{2}$$

where ℓ_{old} is the behavior-policy token log-probability, m is the response-token mask, R is the reward or verifier result, and h identifies the generation, reward, and selection configurations. The record makes an update auditable. In particular, a PPO- or GRPO-style importance ratio compares a candidate policy to the actual behavior policy,

$$\rho_{t(\theta;v)} = \frac{\pi_{\theta(y_t \mid x, y_{<}t)}}{\pi_{\theta_v}(y_t \mid x, y_{<}t)}. \tag{3}$$

When θ drifts far from θ_v, this ratio becomes more variable and a nominally on-policy update becomes a weak approximation. Policy versions, log-probabilities, response masks, and a maximum allowed lag are therefore training data, not incidental logging.

2.1 The Rollout Lifecycle

An online alignment iteration has a compact but consequential lifecycle:

Roll out → score → select → update → refresh version

The prompt distribution may be fixed, sampled from a curriculum, or refreshed from a task pool. The generation configuration—temperature, maximum response length, stop conditions, and any format constraint—changes which trajectories are observed, so it belongs in h in Equation 2. Scoring can be a learned Reward Model, a deterministic checker, a process signal, or a combination, but its version must be equally explicit. Finally, the selected records are transformed into token-level learning signals and used for a bounded number of updates before another rollout round begins.

Exploration is not merely random noise. Sampling temperature, multiple Rollouts per prompt, prompt diversity, and task difficulty determine whether the learner observes useful alternatives. Very low diversity repeatedly trains on one familiar solution pattern; unconstrained diversity can spend most computation on malformed or unscorable trajectories. Chapter 17 explains why multiple candidates can increase both learning signal and inference-time success probability. Here the additional point is that sampling policy and selection policy jointly define the next training distribution.

3 Selection, Filtering, and Iterative Improvement

When rewards are sparse or evaluation is expensive, systems often retain only a subset of sampled responses. Let $w(x, y)$ be a nonnegative selection weight; binary policy filtering uses $w \in \{0, 1\}$. The distribution after filtering is, schematically,

$$p_{\text{selected}(y \mid x)} = \frac{\pi_{\theta_v}(y \mid x) w(x, y)}{\sum_{y'} \pi_{\theta_v}(y' \mid x) w(x, y')}. \tag{4}$$

This expression exposes a trade-off. Rejection-style selection can remove invalid outputs, failed verifier checks, duplicate answers, or obviously low-quality candidates. It can also bias the retained data toward short, easy, stylistically preferred, or already-solved cases. It is therefore not the distribution-preserving accept–reject correction used in exact speculative sampling (Chapter 23). It is a deliberate change to the data distribution and must be evaluated as such.

Verifier-guided selection is particularly attractive when correctness is executable. A mathematics checker, unit-test suite, or symbolic validator can identify candidates that satisfy a declared condition without claiming to judge every dimension of response quality. A learned score can rank candidates where no deterministic verifier exists, but it imports the calibration, style bias, and reward-hacking risks discussed in Chapters 13 and 18. A filter should retain its reason code and score, not only its final accept/reject bit.

3.1 Expert Iteration and Self-Improvement

Expert Iteration is a general pattern in which a current policy produces or helps search for solutions, a stronger selection procedure identifies better behavior, and the policy is trained to imitate or optimize toward that selected behavior. In the original formulation, planning provides the stronger expert signal while a neural policy generalizes it [1]. In language-model reasoning, the selector may instead be a verifier, a search procedure, or a reward rule:

$$\pi_{\theta_v} \rightarrow \text{sample or search} \rightarrow \text{select verified candidates} \rightarrow \text{update } \pi_{\theta_{v+1}}. \tag{5}$$

The loop is not guaranteed to improve itself. If the selector favors a superficial format, the next policy can become better at that format rather than at the task. If generated data lacks unsolved cases, the learner receives little new corrective information. ReST is one example of alternating generation and reward-filtered improvement for language models; it illustrates both the practical appeal of selected self-generated data and the need to state the filter and update regime precisely [2].

4 DAPO: A Concrete On-Policy Design

DAPO (Decoupled Clip and Dynamic sAmpling Policy Optimization) is an open implementation-oriented design for large-scale LLM reasoning reinforcement learning. Its reported formulation builds on group-relative Rollouts, but changes several details that become important for long reasoning responses: dynamic sampling, a decoupled clipping range, a token-level policy-gradient loss, and overlong-response shaping [3]. These choices are useful to study because they make otherwise implicit decisions about which Rollouts enter an update and how individual tokens are weighted.

For a prompt q with answer a, sample a group $(o_i)_{i=1}^{G}$ from an old policy. In the binary-verification setting of the original formulation, define

$$c(q) = \sum_{i=1}^{G} 1[\text{equivalent}(a, o_i)]. \tag{6}$$

Groups with $c(q) = 0$ or $c(q) = G$ have no within-group correctness variation and give zero standardized relative advantage. Dynamic sampling oversamples prompts and retains groups satisfying $0 < c(q) < G$, then refills the batch as necessary. This can concentrate rollout compute on prompts that currently provide a contrastive learning signal. It must not be interpreted as a universal rule: it can underrepresent consistently solved, consistently unsolved, or non-binary-reward tasks.

4.1 Ratios, Asymmetric Clipping, and Group Advantages

For each response token, DAPO uses the behavior-policy ratio

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}. \tag{7}$$

With group rewards R_i, let $\mu_R = \frac{1}{G}\sum_j R_j$ and $s_R = \sqrt{\frac{1}{G}\sum_j (R_j - \mu_R)^2}$. A representative relative advantage is

$$\hat{A}_i = \frac{R_i - \mu_R}{s_R + \varepsilon}. \tag{8}$$

The exact reward and normalization conventions are implementation contracts. In the published binary-reward setting, the same response-level relative advantage is applied to its valid tokens. The clipped local objective is

$$\ell_{i,t}(\theta) = \min\Big(r_{i,t}(\theta)\hat{A}_i, \text{clip}\big(r_{i,t}(\theta), 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}}\big)\hat{A}_i, \Big). \tag{9}$$

PPO commonly presents a symmetric interval around one. DAPO separates the bounds and, in its Clip-Higher design, uses an upper allowance that can be larger than the lower one. Increasing $\varepsilon_{\text{high}}$ permits a larger probability increase for advantageous tokens; keeping ε_{low} relatively small limits large probability decreases. This is a design for maintaining useful exploration, not a proof that asymmetric clipping is always preferable. The ratio and clipping behavior should be monitored alongside KL drift

and entropy even when an implementation does not include the same explicit KL term used by classical RLHF in Chapter 14.

4.2 Token-Level Loss and Long Responses

Let $N_{\text{tok}} = \sum_{i=1}^{G} |o_i|$ count valid response tokens in a group. A token-level group objective can be written as

$$J_{\text{DAPO}(\theta)} = \mathrm{E}\left[\frac{1}{N_{\text{tok}}} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} \ell_{i,t}(\theta)\right]. \tag{10}$$

This differs from first normalizing each response by its own length and then averaging responses equally. Under Equation 10, a longer response contributes more valid token terms. That can give a correct long reasoning trace more influence, but it also makes response-length behavior part of the optimization objective. Token masks must exclude prompt, padding, and post-termination positions; otherwise the apparent change in weighting is an indexing bug rather than an algorithmic choice.

The original DAPO system also uses a shaped length penalty near a maximum rollout length. With a maximum $L_{\max}$ and a soft interval L_{cache}, one representative form is

$$R_{\text{length}(y)} = \begin{cases} 0 & |y| \le L_{\max} - L_{\text{cache}} \\ \setminus \frac{L_{\max} - L_{\text{cache}} - |y|}{L_{\text{cache}}} & L_{\max} - L_{\text{cache}} < |y| \le L_{\max} \\ \setminus -1 & |y| > L_{\max}. \end{cases} \tag{11}$$

The soft region avoids treating every near-limit response identically, while the terminal penalty discourages trajectories that exhaust the length budget. Its purpose is not to make all answers short. A length penalty must be checked against task correctness, because some problems genuinely require a longer derivation.

5 Data Quality, Modes, and Monitoring

Online data quality is conditional on the current policy. A batch with high mean reward can still be low value if it contains duplicates, template exploitation, nearly identical easy prompts, or reward artifacts. Conversely, a low reward batch can be informative if it contains calibrated hard cases and enough variation to form a stable relative signal. Curriculum, prompt-source mixture, sampling configuration, verifier coverage, and selection thresholds should therefore be versioned with every update.

Signal	What it can reveal	Required interpretation
Policy lag and ratios	Stale behavior data or overly large updates.	Read with update epochs, clip fraction, and policy-version spread.
Group reward variance	Whether relative advantages are informative.	Separate all-success and all-failure groups from noisy rewards.
Selection rate	Filter strictness and rollout-compute waste.	Stratify by prompt source, length, and difficulty.
Response length and stop rate	Overlong behavior, truncation, or shortcut responses.	Check jointly with correctness and length shaping.
Entropy, KL, and clip fraction	Exploration loss or excessive policy movement.	Compare against behavior and reference-policy contracts.
Held-out task and regression scores	Whether online reward gains transfer.	Use the multi-objective suite of Chapter 18.

Table 1: Online alignment metrics must be interpreted jointly. No single rollout statistic establishes durable policy improvement.

The principal failure modes are coupled. Overly strict filtering can cause mode collapse; weak filtering can train on invalid outputs. A stale rollout pool can invalidate an assumed on-policy update; excessively fresh but narrow data can overfit one policy mode. A verifier can be objective about a narrow property yet still be exploitable through answer formatting or loopholes. The remedy is not a larger reward number, but a reproducible decomposition: inspect prompt distribution, generation configuration, selection decision, reward or verifier trace, advantage statistics, update size, and held-out behavior in that order.

6 Implementation Contracts

An implementation should treat on-policy alignment as a versioned data pipeline. Each rollout must record its behavior-policy version, token log-probabilities, response mask, prompt identifier, generation configuration, reward or verifier version, and selection reason. The learner must enforce a bounded policy lag and reject or explicitly reweight records outside that contract. Prompt, completion, padding, and terminal tokens need unambiguous masks; only intended completion tokens receive policy-gradient loss.

Group construction must specify group size, reward normalization, zero-variance handling, and whether all-success or all-failure groups are retained. Filters should preserve enough metadata to audit selection bias by source, difficulty, length, language, and reward mode. Length limits, shaped penalties, stop conditions, and answer parsing must be evaluated with correctness rather than optimized as isolated proxy metrics. Finally, online metrics need held-out, human, or deterministic regression evidence as specified in Chapter 18.

7 Summary

On-policy alignment replaces a static training corpus with a moving distribution produced by a recent policy. That change creates a powerful route to verifier-guided learning and iterative policy improvement, but it makes provenance, policy versioning, exploration, selection, and distribution shift first-class algorithmic concerns. Rejection-style filtering and Expert Iteration can turn sampled candidates into stronger supervision only when their selectors are reliable and their biases are measured. DAPO makes four practical choices explicit: retain informative mixed-outcome groups, separate clipping bounds, weight valid tokens directly, and shape overlong responses. These are levers in a coupled online system, not independent recipes. The next systems chapter will address how Rollout, reward, policy, and critic work is organized and scheduled at distributed scale.

8 References

[1] T. Anthony, Z. Tian, and D. Barber, "Thinking Fast and Slow with Deep Learning and Tree Search," *arXiv preprint arXiv:1705.08439*, 2017, doi: 10.48550/arXiv.1705.08439.

[2] C. Gulcehre *et al.*, "Reinforced Self-Training (ReST) for Language Modeling," *arXiv preprint arXiv:2308.08998*, 2023, doi: 10.48550/arXiv.2308.08998.

[3] Q. Yu *et al.*, "DAPO: An Open-Source LLM Reinforcement Learning System at Scale," *arXiv preprint arXiv:2503.14476*, 2025, doi: 10.48550/arXiv.2503.14476.